

CS 486/686

Neural Networks and Learned Embeddings

Yuntian Deng

Lecture 18

From fixed features to learned vector representations

Recorded lecture (asynchronous)



This lecture is **pre-recorded** - I'm away at **ICML** this week, so there's no in-person class today. Watch at your own pace.



Due today (Thu Jul 9): CS686 project proposal. **Assignment 2** is now due **Thu Jul 16**. Chat 9 is out and Assignment 3 is released.



Questions? Post on **Piazza** - the TAs are available, and I'll follow up when I'm back.

The problem with fixed features

L17 assumed the useful input vector already exists.

pixels

Where is the cat ear?

word IDs

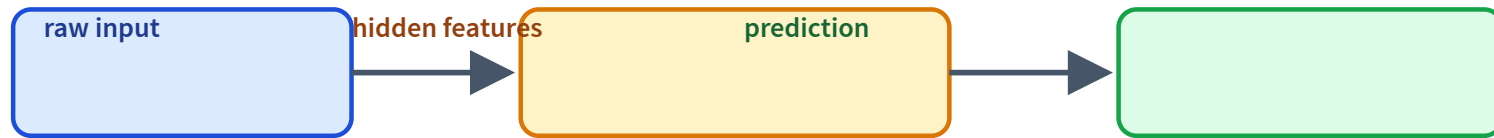
Why is dog close to
puppy?

screenshots

Which region is a
button?

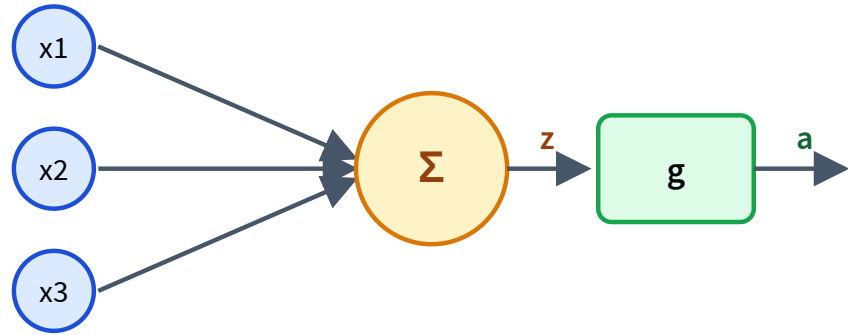
Modern ML learns the features too.

Neural nets learn representations



The hidden layer is not just computation: it is a learned view of the input.

A neuron is a feature detector



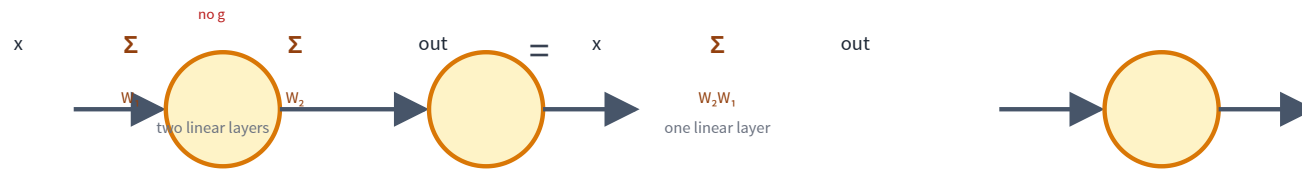
inputs x_1, x_2, x_3

weighted sum: $z = w^\top x + b$

activation: $a = g(z)$

Why add a nonlinearity?

Two linear layers in a row collapse into one:

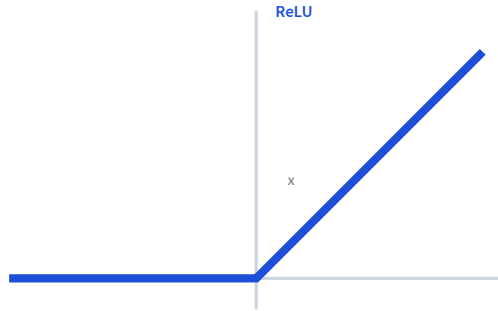


$$W_2(W_1 x) = (W_2 W_1) x$$

Depth alone adds nothing — we need a nonlinearity between the layers.

The fix: a nonlinearity

Insert g between layers: $h = g(W_1x)$, $\hat{y} = W_2h$.



$$g(x) = \max(0, x)$$

The bend is what lets stacked layers represent nonlinear functions.

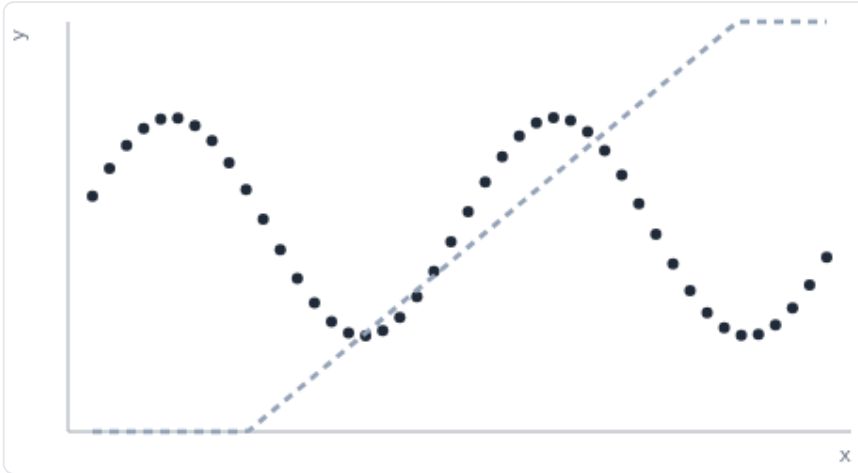
Playground: does nonlinearity matter? LIVE

Interactive demo — runs live in your browser (open the slides to try it).

Model: 1 input $\rightarrow$ 12 tanh hidden units $\rightarrow$ 1 output · target $y = \sin(2x)$ · trained with the L17 loop

activation: OFF

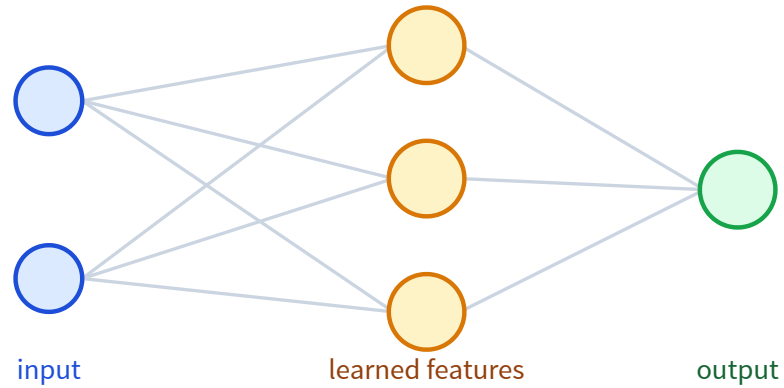
Train



activation: **OFF (linear)**
untrained — press **Train**

Same net, same data. Activation OFF: stacked linear layers stay a line. ON: the layer bends to fit the wave. That bend is what nonlinearity buys.

One hidden layer = learned feature map

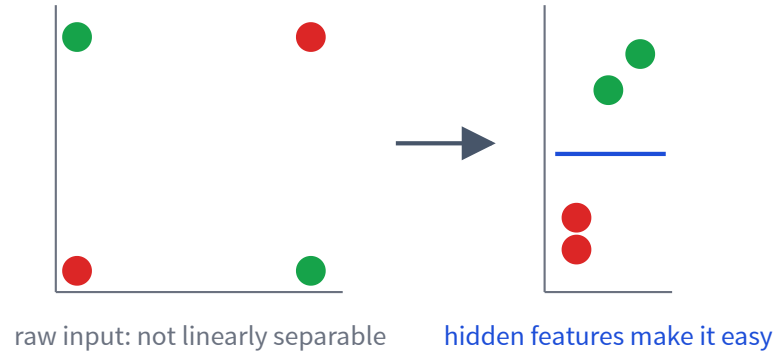


Each hidden unit learns a detector.

The output layer combines detectors.

This is how fixed features become learned features.

XOR: why hidden features help



A hidden layer can transform the space.

Then a simple output layer can solve the task.

Playground: train a net on XOR LIVE

Interactive demo — runs live in your browser (open the slides to try it).

Train

activation: OFF



activation: **OFF**

untrained — press **Train**

○ class 0 ■ class 1

Green/red = the net's decision surface; markers are the true labels. Turn the activation OFF and the 2-2-1 net becomes linear — it cannot separate XOR no matter how long it trains.

An MLP: a bigger model, same loop

$$h = g(W_1x + b_1)$$

$$\hat{y} = \text{softmax}(W_2h + b_2)$$

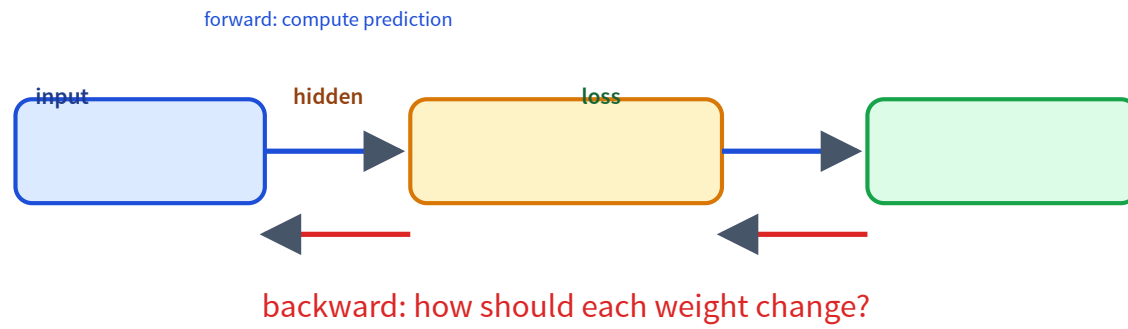
Every operation is differentiable, so L17's training loop applies unchanged.

The code barely changes

```
model = nn.Sequential(  
    nn.Linear(d_in, 64),  
    nn.ReLU(),  
    nn.Linear(64, n_classes),  
)  
  
loss = loss_fn(model(x), y)  
loss.backward()  
optimizer.step()
```

The function is richer; the training pattern is the same.

Backpropagation assigns credit



What do hidden layers learn?



Deep models compose simple features into useful abstractions.

CNNs learn visual features



shallow: curves



middle: parts



deep: objects



one deep neuron = a "ball" detector

Feature visualizations (what each neuron responds to most). Olah et al., "Feature Visualization," Distill 2017 (CC-BY).

Local filters detect small patterns.

Shallow: edges & curves.

Deeper: object parts.

Deepest: whole objects.

Later: VLMs reuse image encoders (L23).

Word IDs have no geometry

IDs are arbitrary labels:

cat = 17, the = 18, dog =
932

id 17 (cat) is no closer to id 18 (the) than
to id 932 (dog) — adjacency means
nothing.

An embedding gives each a vector,
where **distance = similarity**:



Similar meaning → nearby vectors (cat close to dog; "the" is off on its own).

An embedding table is a learnable lookup

It is just a **lookup table** (a dictionary): a token's **id** selects its **row**.

token	id	row = learned vector
cat	17	[0.22, -0.71, 0.10, ...]
dog	932	[0.19, -0.66, 0.12, ...]
car	40	[-0.80, 0.31, -0.40, ...]

`emb[17]` → the "cat" row

```
emb = nn.Embedding(  
    num_embeddings=vocab_size,  
    embedding_dim=128,  
)
```

Unlike a fixed dictionary, the rows are **parameters**: gradient descent updates them during training.

How does an embedding learn?

Words that appear in the **same contexts** get pulled together.

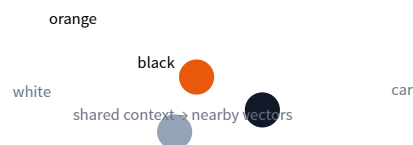
I have a ____ cat — the blank is almost always a color:

orange

black

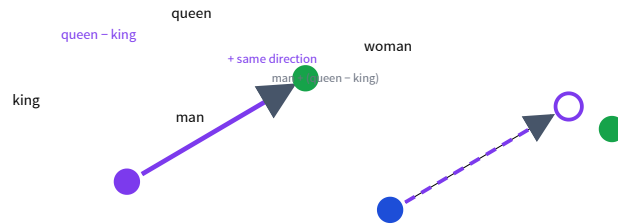
white

To fill the same blanks well, the model must give these color words **similar embeddings**.



Directions can carry meaning

$$\text{king} - \text{man} + \text{woman} \approx \text{queen}$$



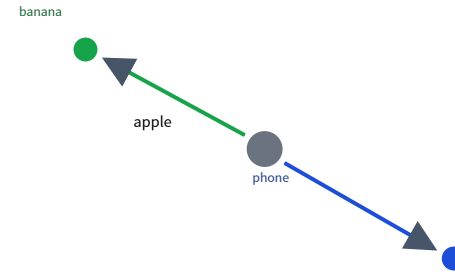
Add the same "royal" direction to **man** and you land *near* woman (not exactly).
Standalone embeddings — word2vec (Mikolov 2013), GloVe (Pennington 2014) —
famously captured such regularities.

Today embeddings are mostly learned **implicitly inside LLMs**; modern contextual vectors
rarely do clean arithmetic.

One word can mean two things

"I ate an **apple**" → **fruit**

"**Apple** released a phone" →
company



A single fixed vector for "apple" cannot be both.

Modern models embed the **whole sentence**, so a word gets a different **contextual** vector each time.

Inspect embeddings by projecting them LIVE

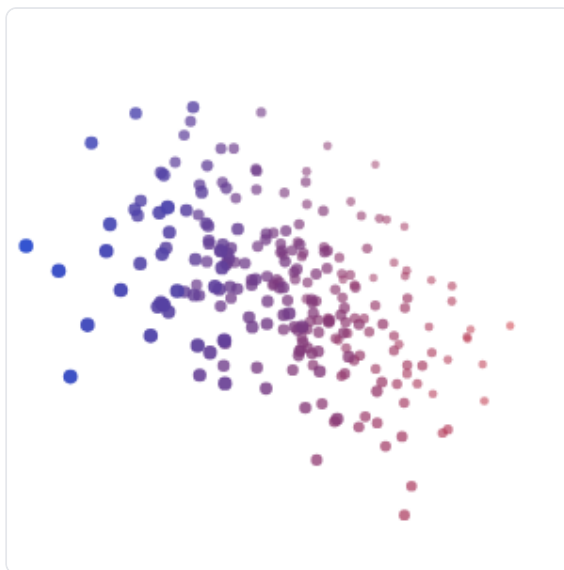
PCA squeezes many dimensions to 2D by keeping the directions of most spread. — drag the cloud on the live slides.

Drag the 3D cloud to rotate. PCA finds the flat plane of most spread and drops the thin 3rd direction.

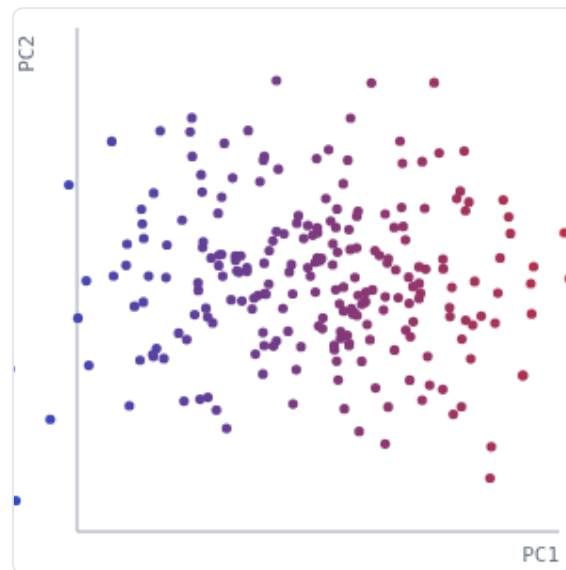
Flatten to 2D

Reseed

3D data (drag to rotate)



PCA → 2D



PCA on real data: a million chats

LIVE DATA

Real WildChat conversations, 1536-dim embeddings projected to 2D — hover to read one, click to open it.

Real WildChat conversations. Each is a 1536-dim vector (OpenAI text-embedding-3-small), projected to 2D with PCA. Hover a dot to read it; click to open it.

Reset view

[Explore it live at wildvisualizer.com](https://wildvisualizer.com) ↗



1428 real conversations — hover to read, click to open.

Playground: embed real words

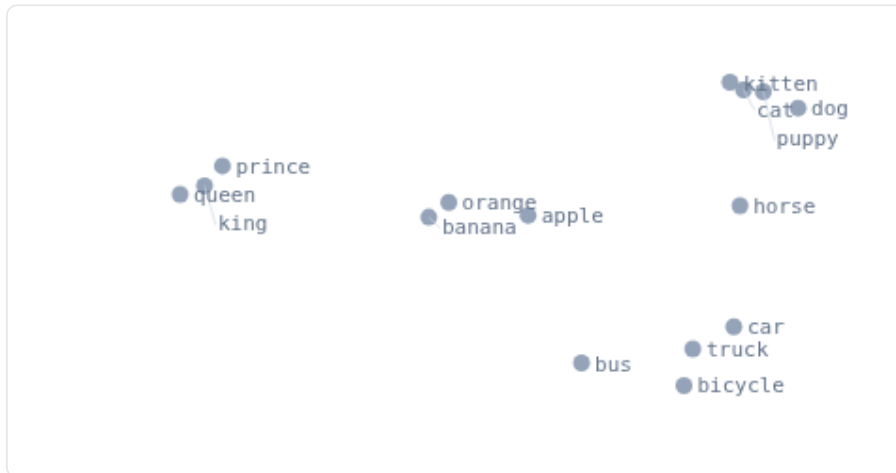
LIVE MODEL

Interactive demo — a real embedding model runs in your browser (open the slides to try it).

dog, puppy, cat, kitten, horse, car, truck, bus, bicycle, apple, banana, orange, king, queen, prince

Embed

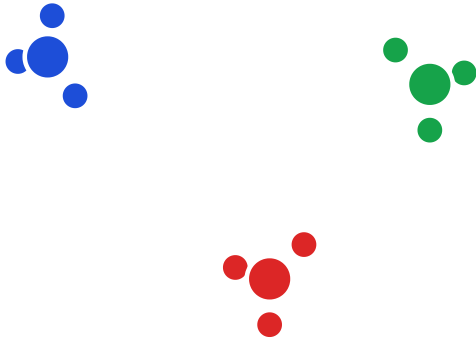
Reset words



click a point to see its
nearest neighbors

Type any words or short sentences. MiniLM computes a contextual embedding (not a fixed lookup), then PCA squeezes it to 2D. Similar meanings land near each other.

Cluster embeddings with k-means



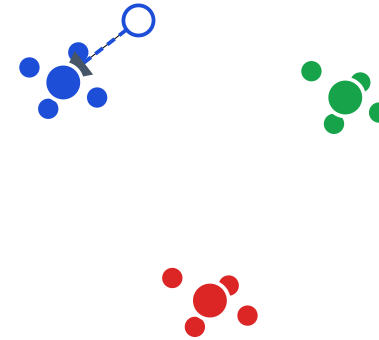
After embeddings exist, k-means becomes practical.

- inspect datasets
- find groups or duplicates
- build codebook intuition

The k-means algorithm

1. Choose k ; initialize k centroids (e.g. random data points).
2. Repeat until assignments stop changing:
 - a. **Assign** — put each point with its nearest centroid.
 - b. **Update** — move each centroid to the mean of its points.

assign = color by nearest · update = move to mean



Converges to a **local** optimum — depends on initialization, so run a few times and keep the best.

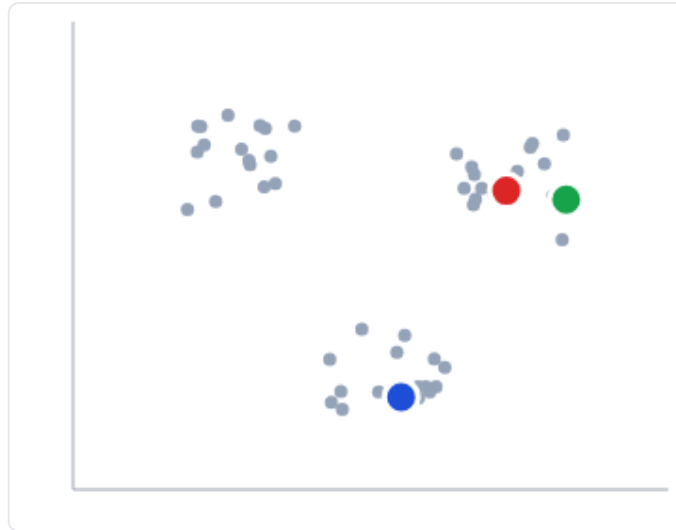
Playground: k-means, step by step LIVE

Interactive demo — runs live in your browser (open the slides to try it).

Step

Run

Reseed

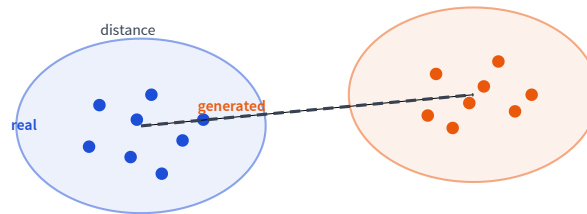


iteration 0
centroids placed

Each step: assign every point to its nearest centroid, then move each centroid to its members' mean. Repeat until nothing moves.

Measuring image quality: FID

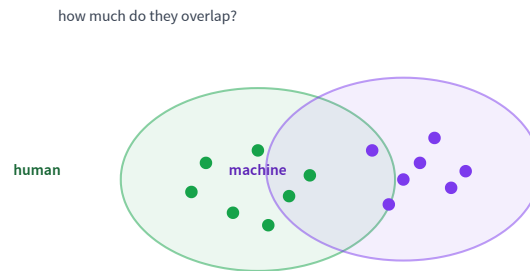
Embed real and generated images with an Inception network, then compare the two **distributions** in feature space.



FID = distance between the two feature distributions — lower means the generations look more real.

Measuring text quality: MAUVE

Same idea for text: embed human and machine text in a language model's space and compare the distributions.



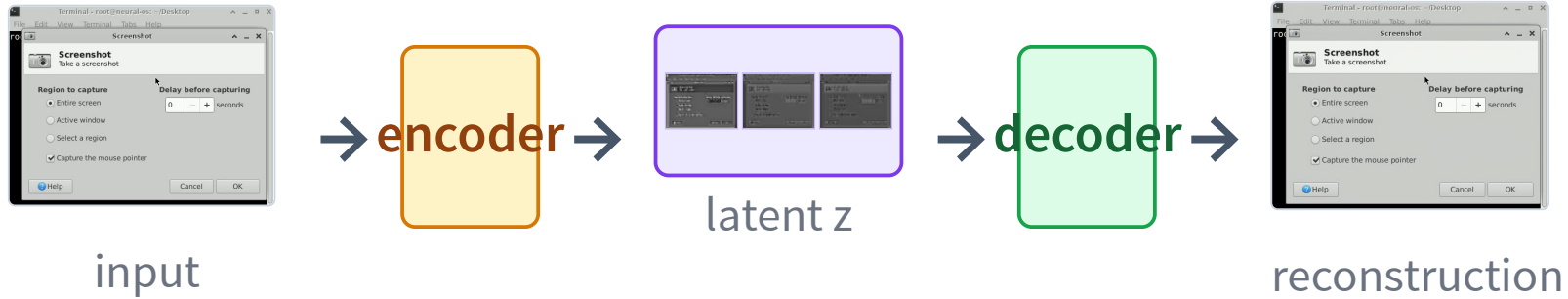
MAUVE summarizes how much the two distributions overlap — higher means more human-like.

Autoencoders learn compression



Latent diffusion will generate in a compressed latent space.

A real autoencoder: NeuralOS



The decode looks the same as the input — the small latent kept everything that matters.

Autoencoder from NeuralOS (neural-os.com).

Why compress? To make generation feasible



neural-os.com

Generating in pixel space is expensive.

$$512 \times 384 = 196,608 \text{ pixels}$$

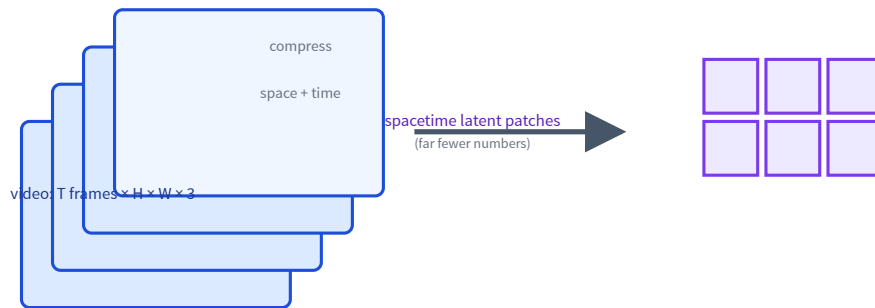
$$\rightarrow 64 \times 48 = 3,072 \text{ pixels}$$

$\approx 64\times$ fewer pixels to generate.

NeuralOS predicts the next frame in this small latent — a neural operating system.

Video: compress space and time

A video is many frames, so pixel cost explodes — compress the time axis too.

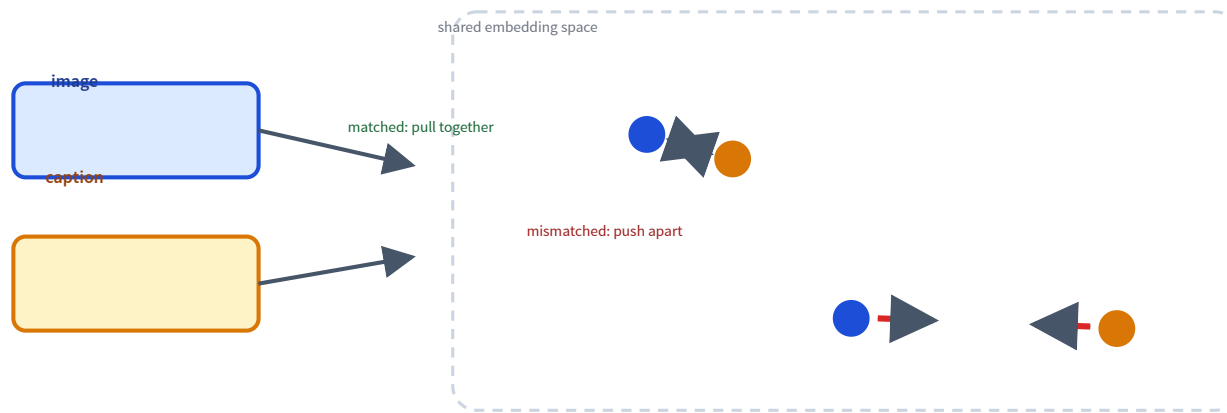


Sora compresses space and time into spacetime latent patches, then generates there.

OpenAI, "Video generation models as world simulators," 2024.

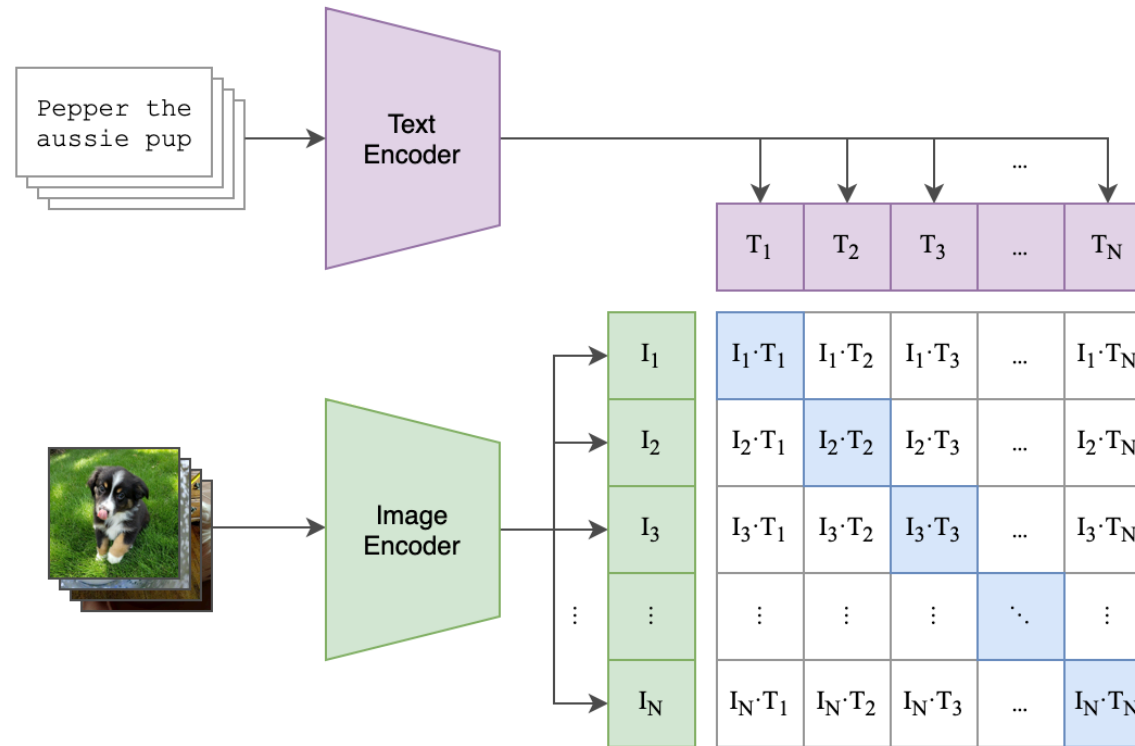
Contrastive learning aligns modalities

Put images and their captions into one shared vector space.



CLIP: contrastive pre-training

(1) Contrastive pre-training



Across a batch, the matching image-text pairs (the diagonal) are trained to score highest.

What a shared image-text space unlocks

zero-shot classification

score an image against prompts like "a photo of a {label}" — no task-specific training

retrieval

search images with text (and vice versa) by nearest vectors in the shared space

This shared space is the bridge to vision-language models (L23).

Why this matters for LLMs

tokens

become embeddings

attention

moves information
between
embeddings

transformers

scale this up

Recap

- ✓ Neural nets learn **features**, not just predictions.
- ✓ Backprop assigns credit through the graph.
- ✓ **Embeddings** are learned vectors.
- ✓ PCA, t-SNE, and k-means help inspect embeddings.
- ✓ Autoencoders and CLIP prepare us for diffusion and VLMs.

Next question

A sentence is a sequence of embeddings.

How should each word look at the other words?

L19: sequence models, attention, and transformers.